a performance point of view, for example; this can happen for some complex queries. Or you might choose to use a micro-ORM, like Dapper (https://github.com/DapperLib/Dapper), which requires you to write your own SQL throughout. Whatever the reason, let's assume you are using SQL to retrieve the city details in the *City.cshtml.cs* page. Remember that the filter value used to query the database is extracted for the URL. This is considered *user input* because the user can access and modify it. Think back to the section on binding sources in chapter 5 in which we looked at model binding. All of those binding sources—query string, form, route values, headers, and even cookies are potential sources of user input.

On the City page, your SQL statement might look something like the following, where you incorporate the value of the binding target, the `Name` property, into your query:

```
select * from cities where name = '" + Name + "';
```

If the value passed in the URL is `paris`, the resulting SQL statement will be

```
select * from cities where name = 'paris';
```

Now if the URL was altered so that the route value becomes `paris';drop table aspnetusers;--`, the resulting SQL will be

```
select * from cities where name = 'paris';drop table aspnetusers;--';
```

This is a perfectly valid SQL statement that will result in the city named Paris being retrieved from the database, followed by the user table being deleted in any database system that accepts bulk statements—which is most of them. Chances are you will notice this pretty quickly because the application will crash, and hopefully you will be able to restore from a backup, but you will likely lose data. But how about the following example, which results in all prices being set to 30% of their original value? How long will it take you to notice that users have been busy booking ultra-cheap vacations?

```
select * from cities where name = 'paris';
➥ update properties set price = price * 0.3;--';
```

This vulnerability occurs because there is no sanitizing of the user input undertaken. It is incorporated into the dynamically generated SQL statement as is. The solution for preventing SQL injection is to ensure nothing can creep into the SQL that might change the